

Exercise 1

PAGE No.

DATE

Counting basic operations

1. Consider the following statement:

```
cout << "Hello";
```

How many times is the statement executed?

→ The statement executed only one time. Because no multiline statement or loop are used.

1 times

2. Consider the code:

```
for (int i = 0; i < n; i++)  
    cout << i;
```

- a) How many times does the cout statement executed when $n = 10$?

→

i is start from 0 to 9 so total values are 10 and $n = 10$ cout statement executes 10 times

- b) Consider How many times when $n = 100$?

→

i is start 0 to 99 and $n = 100$ so total value are 100 cout statement executes 100 times

3. Consider the following code

```
for (int i = 0; i < n; i++) {
    int x = i + 1;
    int y = x * 2;
}
```

if $n = 5$, how many assignment operations are performed?

→ Assignment operator means give value to variable in this code

1) $\text{int } i = 0 \rightarrow 1$ assignment operation

2) $\text{int } x = i + 1; \rightarrow 1$ assignment operation
and $n = 5$ is for assignment operations

3) $\text{int } y = x * 2; \rightarrow 1$ assignment operation and $n = 5$ is for assignment operations

4) $i++ \rightarrow 5$ assignment operations

Total will be $= 1 + 5 + 5 + 5 = 16$

in this code 16 assignment operations are performed.

4. consider loop:

```
for (int i = 0; i < n; i += 2)
    cout << i;
```

a) How many times does the loop run when $n = 10$?

in this loop i starts from $i = 0$ or i increases by 2
 $i += 2$ and loop runs $i < n$
 So the value of i is 0, 2, 4, 6, 8
 Total value = 5

The loop runs 5 times

b) Approximately how many times does it run for general n ?

- 1) The loop starts from $i = 0$
 2) and it increases by $i += 2$
 (2 each time)
 3) it stops when $i \geq n$

So the number of iterations is

$$\text{Number of iterations } \frac{n}{2}$$

For large n , the loop runs about $\frac{n}{2}$ times.

5. Consider the loop

```
for (int i = n; i > 0; i--)
    cout << i;
```

does counting down from n instead of up to n change the total amount of work?

5) Consider loop

```
for (int i = n; i > 0; i--)  
    cout << i;
```

Does counting down from n instead of up to n change the total amount of work?

- The loop runs n times (for $i = n, n-1, \dots, 1$)
 - Counting down instead of up doesn't change the number of times the loop runs
- Same amount of work, just the numbers go in reverse

Nested loops and Growth

6. Consider the code:

```
for (int i=0 ; i<n ; i++)  
    cout << i;
```

```
for (int j=0 ; j<n ; j++)  
    cout << j;
```

How many print total print statements are ~~exec~~ executed?

→ 1) First loop

i goes from 0 to $n-1$
That's n iterations

So the First loop executes `cout << i`
 n times

2) Second loop

j goes from 0 to $n-1$
That's n iterations

so the second loop executes `cout << i`
`cout << j` n times

total print statements = $n+n = 2n$

So $2n$ print statements.

7. Consider the nested loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        cout << i << j;
```

a) How many times does cout executes
when $n = 3$?

→

When $n = 3$

outer loop runs 3 times ($i = 0, 1, 2$)
inner loop runs 3 times for each i
($j = 0, 1, 2$)

Total prints = $3 \times 3 = 9$

So cout executes 9 times.

b) Table Represent the execution as a table

→ i	j	output
0	0	0 0
0	1	0 1
0	2	0 2
1	0	1 0
1	1	1 1
1	2	1 2
2	0	2 0
2	1	2 1
2	2	2 2

9 prints in order of 00 01 02 10 11 12
20 21 22

8) Loop
Consider the code

```
For (int i = 0; i < n; i++)  
    For (int j = 0; j < 5; j++)  
        cout << i << j;
```

For large n , which loop contributes more to the total work?

→ Outer loop runs n times
inner loop always runs 5 times

So total work = $5n$

For large n , the outer loop contributes more to the total work

9th question after 10th question

10) Consider the code:

```
For (int i = 0; i < n; i++)  
    For (int j = 0; j < n; j++)  
        For (int k = 0; k < n; k++)  
            cout << i << j << k;
```

does adding a loop with a small constant limit significantly change the growth?

→ outer loop (i) runs n times
middle loop (j) runs n times
inner loop (k) runs 3 times (constant)

$$\text{Total work } (n \times n \times 3 = 3n^2)$$

No, it does not change significantly
change the growth
3 is constant

Constants are ignored in time complexity

Growth is still $O(n^2)$

9) Consider the code :

```
For (int i = 0; i < n; i++)  
  For (int j = 0; j < i; j++)  
    cout << "*";
```

a) Draw the output pattern when $n=4$

```
*  
* *  
* * *  
* * * *
```

b) Is the total work closer to n , n^2 ,
or something in between?

Number of ~~s~~ printed

$$1 + 2 + 3 + 4 = 10$$

in general

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

So total work is closer to n^2

Time complexity $O(n^2)$

Input Size and Growth Intuition

11. A program takes 1 second for input size $n = 100$ and approximately 4 seconds for input size $n = 200$. Is the growth more likely linear or quadratic? Explain

input size doubles: $100 \rightarrow 200$

time increases: 1 second $\rightarrow$ 4 seconds

if growth were linear, time would roughly double (1 $\rightarrow$ 2 seconds)

But it increased 4 times, which matches quadratic growth

$$(2n)^2 = 4n^2$$

The growth is more likely quadratic $O(n^2)$

12) Rank the following functions according to their growth rate as n becomes large

$$n, n+100, 2n$$

Functions

- n
- $n+100$
- $2n$

As n becomes large:

- constant don't matter much

Ranking (Slowest $\rightarrow$ Fastest):

$$n \approx n+100 \approx 2n$$

All growth linearly ($O(n)$)

13) Rank the following functions according to their growth rate

$$n, n^2, n^3$$

Use values such as $n=10, 100$, and 1000 to justify your answer

n	n	n^2	n^3
10	10	100	1,000
100	100	10,000	1,000,000
1000	1000	1,000,000	1,000,000,000

Ranking (Slowest $\rightarrow$ Fastest)

$$n < n^2 < n^3$$

14) which function grows faster for larger n ?
 $1000n$ or n^2

Approximately for what value of n does n^2 become larger?

• $1000n$ - linear
 n^2 - quadratic

For small n , $1000n$ can be larger
 But for larger n , quadratic always wins

$$n^2 > 1000n \Rightarrow n > 1000$$

n^2 grows faster
 n^2 becomes larger at about $n \approx 1000$

15) Introduction of Big-O Notation

15 consider the loop

```
for (int i = 0; i < 3 * n + 5; i++)  
    cout << i
```

For large n , which part of the expression mainly determines the number of iterations?

$3n$ grows with n
 $+5$ is a constant
For large n , the constant doesn't matter
For large n .

The $3n$ part mainly determines the
number of iterations
Big- O : $O(n)$

16) Identify the dominant term for large n

$$n^2 + 10n + 50, \quad n + \log n$$

$$n^2 + 10n + 50$$

n^2 grows the fastest

Dominant term : n^2

$$n + \log n$$

Linear growth is faster than logarithmic growth

So Dominant term : n

17) Match each type of code to its Big- O complexity :

Options: $O(n^2)$, $O(1)$, $O(n)$

- A loop that runs a fixed number of times

$O(1)$ Big- O

- A loop runs that runs n times

Big- O - $O(n)$

- Two nested loops each running n times

Big- O : $O(n^2)$

18) Consider the code:

```
For (int i=0; i<n; i++)  
    if (arr[i] == key)  
        break;
```

a) When does the loop terminate early?

- when the key is found before reaching the end of the array
- Best case: the key is at the first position ($arr[0]$)

b) when does it execute all iterations?

- when the key is not present in the array
- Or when the key is at the last position ($arr[n-1]$)